

Code : Root component (App.svelte)

Project reference: STN22_Thoracic belt for respiratory rate measurement

Authors: Hakkou Dounia

Date: 2025–2026 © SIS TEAM NANCY

File header

```
<!--
  Description (EN): Root Svelte component for the RespiraMed PWA.
  Orchestrates the entire application: navigation between tabs, global state
  (selected patient, dark mode, sidebar), dashboard statistics polling,
  and composition of child components PatientForm, PatientList,
  MeasurementDisplay.
  This is the only component mounted directly into the DOM via main.ts.

  Project reference: STN22_Thoracic belt FR
  Authors: Hakkou Dounia
  Date: 2025–2026
  © SIS TEAM NANCY
-->
```

Script block (TypeScript)

```
<script lang="ts">
  // onMount: Svelte lifecycle hook – runs once after the component is
  inserted
  import { onMount } from 'svelte';

  // Import the Patient type from api.ts (id and created_at are required)
  import type { Patient } from '../lib/api';

  // Import child components rendered conditionally inside the main content
  area
  import PatientForm      from '../lib/PatientForm.svelte';
  import PatientList      from '../lib/PatientList.svelte';
  import MeasurementDisplay from '../lib/MeasurementDisplay.svelte';

  // API base URL: prefer the Vite env variable, fall back to production URL
  const API_URL: string = import.meta.env.VITE_API_URL
    ?? 'https://sisteamnancy.fr/api-respiramed';

  // api<T>()
  // Local fetch helper used only by App.svelte to load dashboard statistics.
  // Unlike apiRequest() in api.ts, it normalises the path prefix ('/api').
  // Also handles HTTP 204 (No Content) by returning null instead of parsing
  // an empty body, which would throw a SyntaxError.
  async function api<T>(path: string, options: RequestInit = {}): Promise<T>
  {
    // Ensure path starts with '/api' to avoid double-prefix bugs
    const cleanPath = path.startsWith('/api') ? path : `/api${path}`;
    const url = `${API_URL}${cleanPath}`;
    try {
      const res = await fetch(url, {
        ...options,
        headers: { 'Content-Type': 'application/json', ...(options.headers ||
  {})}
      });
    }
    if (!res.ok) {
      const errorText = await res.text();
      throw new Error(errorText || `Erreur HTTP ${res.status}`);
    }
  }
```

```

    // HTTP 204 means success with no body, return null to avoid JSON parse
error
    if (res.status === 204) return null as unknown as T;
    return await res.json();
  } catch (err) {
    console.error(`Erreur API sur ${url}:`, err);
    throw err;
  }
}

// Reactive state variables

// Patient selected for measurement; null when no patient is active
let selectedPatient: Patient | null = null;

// Controls visibility of the PatientForm in the 'patients' tab
let showForm = false;

// Currently active navigation tab
let activeTab: 'dashboard' | 'measure' | 'patients' = 'dashboard';

// Controls whether the sidebar is expanded (true) or collapsed to icon-
only (false)
let sidebarOpen = true;

// Toggles dark mode; also adds/removes 'dark-mode' class on document.body
let darkMode = false;

// Dashboard statistics object populated by loadRecentData()
let stats = {
  totalPatients: 0, // Total number of patients in the database
  activeSessions: 0, // Number of sessions currently running
  todayMeasurements: 0, // Measurements recorded in the last 24 hours
  avgRPM: 0 // Average respiratory rate across recent
measurements
};

// loadRecentData()
// Fetches dashboard statistics from GET /api/stats.
// Casts all values to Number() to handle cases where the API returns
strings.
// Errors are caught silently so the UI keeps working even if stats fail.
async function loadRecentData() {
  try {
    const data = await api<any>('/stats');
    stats = {
      totalPatients: Number(data.totalPatients || 0),
      activeSessions: Number(data.activeSessions || 0),
      todayMeasurements: Number(data.todayMeasurements || 0),
      avgRPM: Number(data.avgRPM || 0),
    };
  } catch (err) {
    console.error('Impossible de charger les statistiques');
  }
}

// handlePatientAdded()
// Called when PatientForm emits the 'success' event (patient created).
// Hides the form, refreshes stats, and navigates to the Patients tab.
function handlePatientAdded() {
  showForm = false;
  loadRecentData();
  activeTab = 'patients';
}

```

```

// handleSelectPatient()
// Called when PatientList emits the 'selectPatient' custom event.
// Stores the chosen patient and switches to the Measure tab.
function handleSelectPatient(event: CustomEvent<Patient>) {
  selectedPatient = event.detail; // The Patient object is in event.detail
  activeTab = 'measure';
}

// toggleSidebar() / toggleDarkMode()
function toggleSidebar() { sidebarOpen = !sidebarOpen; }

function toggleDarkMode() {
  darkMode = !darkMode;
  // Apply or remove 'dark-mode' class on <body> to trigger global CSS
  rules
  document.body.classList.toggle('dark-mode', darkMode);
}

// onMount()
// Runs once after the component is inserted into the DOM.
// Sets the browser tab title, loads stats immediately, then schedules
// a 15-second polling interval. The returned cleanup function is called
// by Svelte when the component is destroyed, preventing memory leaks.
onMount(() => {
  document.title = 'RespiraMed - Dashboard';
  loadRecentData();
  // Refresh dashboard stats every 15 s without requiring a page reload
  const interval = setInterval(loadRecentData, 15000);
  return () => clearInterval(interval); // Svelte cleanup on component
  destroy
});
</script>

```

HTML template (Svelte)

```

<!-- Root layout: two flex children - sidebar + main container -->
<!-- class:sidebar-closed: reduces sidebar width to 70px when closed -->
<!-- class:dark: activates dark-mode CSS rules inside this subtree -->
<div class="app-layout" class:sidebar-closed={!sidebarOpen}
class:dark={darkMode}>

  <!-- SIDEBAR -->
  <aside class="sidebar" class:open={sidebarOpen}>
    <div class="sidebar-header">
      <div class="brand">
        <div class="brand-icon"></div>
        <#if sidebarOpen>
          <!-- Brand text only visible when sidebar is expanded -->
          <div class="brand-text">
            <h1>RespiraMed</h1>
            <p>Mesure physiologique</p>
          </div>
        </if>
      </div>
    </div>

    <!-- Navigation buttons: clicking sets activeTab -->
    <nav class="sidebar-nav">
      <button class="nav-item" class:active={activeTab==='dashboard'}
on:click={()=>activeTab='dashboard'}>
        Tableau de bord
      </button>
      <button class="nav-item" class:active={activeTab==='patients'}
on:click={()=>activeTab='patients'}>

```

```

        Patients
      </button>
      <!-- Measure tab: disabled until a patient is selected -->
      <button class="nav-item" class:active={activeTab==='measure'}
on:click={()=>activeTab='measure'} disabled={!selectedPatient}>
        Mesure {selectedPatient ? '(Active)' : ''}
      </button>
    </nav>

    <div class="sidebar-footer">
      <!-- Toggle between light and dark mode -->
      <button class="theme-btn" on:click={toggleDarkMode}>
        {darkMode ? 'Mode Clair' : 'Mode Sombre'}
      </button>
    </div>
  </aside>

  <!-- MAIN CONTAINER -->
  <div class="main-container">
    <!-- Topbar: hamburger toggle + dynamic title + active patient badge -->
    <header class="topbar">
      <button class="menu-toggle" on:click={toggleSidebar}>≡</button>
      <h2>
        {#if activeTab==='dashboard'}Vue d'ensemble
        {:else if activeTab==='measure'}Mesure en temps réel
        {:else}Gestion Patients{/if}
      </h2>
      <!-- Show the active patient's name with a pulsing green dot -->
      {#if selectedPatient}
        <div class="active-patient-tag">
          <span class="dot"></span> {selectedPatient.prenom}
{selectedPatient.nom}
        </div>
      {/if}
    </header>

    <!-- Main content: conditional rendering based on activeTab -->
    <main class="content">
      {#if activeTab === 'dashboard'}
        <!-- Dashboard: 3 stat cards loaded from /api/stats every 15 s -->
        <div class="stats-grid">
          <div class="stat-card"><h3>Total Patients</h3><p class="stat-
value">{stats.totalPatients}</p></div>
          <div class="stat-card"><h3>Mesures (24h)</h3><p class="stat-
value">{stats.todayMeasurements}</p></div>
          <div class="stat-card"><h3>Sessions actives</h3><p class="stat-
value highlight">{stats.activeSessions}</p></div>
        </div>

        {:else if activeTab === 'measure'}
          <!-- Measure tab: requires a selected patient to render
MeasurementDisplay -->
          {#if selectedPatient}
            <MeasurementDisplay patient={selectedPatient} />
          {:else}
            <div class="empty-state">
              <p>Veuillez sélectionner un patient dans l'onglet "Patients" pour
commencer.</p>
              <!-- Shortcut button: navigates directly to the Patients tab -->
              <button on:click={() => activeTab = 'patients'}>Voir la
liste</button>
            </div>
          {/if}

          {:else if activeTab === 'patients'}

```

```

    <!-- Patients tab: toggle between PatientList and PatientForm -->
    <div class="patients-header">
      {#if !showForm}
        <!-- Add button: shows PatientForm in creation mode -->
        <button class="btn-primary" on:click={()=>showForm=true}>+
Ajouter un patient</button>
      {/if}
    </div>

    {#if showForm}
      <!-- PatientForm: on:success calls handlePatientAdded(), on:cancel
      hides the form -->
      <div class="form-container">
        <PatientForm on:success={handlePatientAdded}
on:cancel={()=>showForm=false}/>
      </div>
    {/else}
    <!-- PatientList: on:selectPatient calls handleSelectPatient() -->
    <PatientList on:selectPatient={handleSelectPatient}/>
  {/if}
</main>
</div>
</div>

```

CSS styles (global + scoped)

```

<style>
  /* Global body reset: applied to all elements via :global() */
  :global(body) { margin:0; font-family: 'Segoe UI', Tahoma, Geneva, Verdana,
sans-serif;
    background:#f0f2f5; transition: background 0.3s; }
  /* Dark mode body override: activated by toggleDarkMode() */
  :global(body.dark-mode) { background: #1a1a1a; color: white; }

  /* Flex layout: sidebar on the left, main container fills the rest */
  .app-layout { display:flex; min-height:100vh; }

  /* Sidebar: fixed width 260px, collapses to 70px when .sidebar-closed is
  active */
  .sidebar { width:260px; background:#fff; border-right:1px solid #ddd;
    display:flex; flex-direction:column; transition: width 0.3s; }
  .sidebar-closed .sidebar { width: 70px; }

  /* Active nav item: purple-to-blue gradient matching the app primary color
  */
  .nav-item.active { background: linear-gradient(135deg, #7c3aed 0%, #93c5fd
100%);
    color: white; }
  /* Disabled nav item: greyed out, cursor changes to indicate non-
  interactive */
  .nav-item.disabled { opacity: 0.5; cursor: not-allowed; }

  /* Stat card: white box with subtle shadow and hover lift transition */
  .stat-card { background: white; padding: 25px; border-radius: 12px;
    box-shadow: 0 2px 6px rgba(0,0,0,0.08); text-align: center; }
  .stat-value { font-size: 36px; font-weight: bold; color: #2c3e50; }
  .stat-value.highlight { color: #e67e22; } /* Orange for active sessions */

  /* Patient tag: pulsing green dot + name badge in the topbar */
  .active-patient-tag { background: #e8f4fd; color: #2980b9;
    padding: 8px 16px; border-radius: 20px; font-size: 14px; }
  .dot { width: 8px; height: 8px; background: #27ae60; border-radius: 50%;
    animation: pulse 2s infinite; }

```

```
@keyframes pulse { 0%, 100% { opacity: 1; } 50% { opacity: 0.5; } }

/* Dark mode overrides for sidebar, topbar, and stat cards */
.dark .sidebar, .dark .topbar, .dark .stat-card { background: #2d2d2d;
  border-color: #404040; color: white; }
</style>
```